

Let the optimal path start in x and end in y . We call the shortest path from x to y the primary path, and we call the subtrees that remain when we remove the primary path secondary subtrees. In optimal solution, we will go through the primary path, and solve secondary subtrees along the way (turn all bulbs on). It's obvious that it's optimal to solve a secondary subtree when we are on the primary node that connects to it, it doesn't pay off to return to it later.

We will now describe the algorithm for solving a secondary subtree, rooted in its primary node. When we are done with the subtree, we need to be in the root, to be able to exit. Since each move changes the parity of the number of bulbs that are off, and each edge will be traversed an even number of times, we can see that the parity of the number of bulbs that will be off in the end is determined. If it's even, it's optimal to turn all bulbs on, and if it's odd, it's optimal to have root turned off, and all others turned on.

That brings us to the following algorithm: we recursively solve subtrees of child nodes, and if some child remains off, we move down and up (that will flip the child and the current node). We do the same thing

when moving along the primary path. It's easy to see that this gives the minimal number of steps. If y is turned off in the end, then there is no solution that starts in x and ends in y .

We will use dynamic programming to solve the problem. Root the initial tree in any node. Let $dp[i][j][k]$ be the minimal cost (number of nodes in the sequence), where i is the current node, j is 0 if when after solving the subtree the node i will be off, and 1 if it will be on, and k marks if the primary path goes through i .

For the case $k = 0$ (without the primary path), we just sum up the dp states of children with $k = 0$ and add the necessary number of crossings of edges between i and its children. We also need to keep track of the number of times we flipped node i , so we know its final state.

For the case $k = 1$ we have two possibilities, we can take over the primary path from some child, or start the primary path in node i . The transition is the sum of dp states of children with $k = 0$ for all except one child (that has $k = 1$), or the sum when all children have $k = 0$.

For each primary path we can find a node such that the path goes through it, and both endpoints belong to its subtree. We can try for each node to be such a node. One possibility is to take its dp for $k = 1$ and add the number of steps needed to solve the part of the tree "above" (i.e. the whole tree minus its subtree), in this case the node is the endpoint of the primary path. Second possibility is to take two dp -s of children that have $k = 1$, and others to have $k = 0$.

Total complexity is $O(n)$.